

SKILL-09

1. To Implement how to List Active Jobs, Display Status Information, Retrieve Job IDs, Format Output, Update State Information, Test Job Listings

CODE:

```
GNU nano 7.2 jobs_list.c *
#include <stdio.h>
#include <string.h>

struct Job
{
    int jobid;
    int pid;
    char status[20];
};

int main()
{
    struct Job jobs[3] = {
        {1, 1010, "Running"},
        {2, 1011, "Stopped"},
        {3, 1012, "Completed"}
    };

    printf("\nActive Jobs List\n");
    printf("-----\n");
    printf("Job ID\tPID\tStatus\n");
    printf("-----\n");

    for(int i = 0; i < 3; i++)
    {
        printf("%d\t%d\t%s\n",
            jobs[i].jobid,
            jobs[i].pid,
            jobs[i].status);
    }

    return 0;
}
```

RESULT:

```
root@SRIVALLI:~# nano jobs_list.c
root@SRIVALLI:~# gcc jobs_list.c -o jobs_list
root@SRIVALLI:~# ./jobs_list

Active Jobs List
-----
Job ID  PID    Status
-----
1       1010   Running
2       1011   Stopped
3       1012   Completed
root@SRIVALLI:~#
```

REPORT:

In this experiment, a program was developed to list active jobs and display their status information. Each job was assigned a job ID, process ID, and status. The program retrieved and displayed job details in a formatted manner, helping to

monitor and manage job execution. This experiment helps in understanding job listing, status tracking, and job management in Linux systems.

2. To design how to Identify Target Job, Transfer Terminal Control, Resume Processes, Wait for Completion, Update Job State, Test Foreground Switching

CODE:

```
GNU nano 7.2 foreground_job.c *
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t pid;

    pid = fork();

    if(pid == 0)
    {
        printf("Child Process Running...\n");
        sleep(5);
        printf("Child Process Completed\n");
    }
    else
    {
        printf("Foreground Job PID: %d\n", pid);
        printf("Waiting for job completion...\n");
        wait(NULL);
        printf("Job State Updated: Completed\n");
    }

    return 0;
}
```

RESULT:

```
root@SRIVALLI:~# nano foreground_job.c
root@SRIVALLI:~# gcc foreground_job.c -o foreground_job
root@SRIVALLI:~# ./foreground_job
Foreground Job PID: 590
Waiting for job completion...
Child Process Running...
Child Process Completed
Job State Updated: Completed
```

REPORT:

In this experiment, a foreground job was created using the fork() system call. The parent process identified the target job, waited for its completion using wait(), and updated the job state after execution. This experiment helps in understanding foreground job control, process synchronization, and job state management in Linux systems

3. To Analyze Signal Delivery, Understand Signal Masks, Review Process Groups, Explore Terminal Signals, Test Signal Behavior

CODE:

```
GNU nano 7.2 signal_test.c
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void signal_handler(int sig)
{
    printf("\nReceived Signal: %d\n", sig);
}

int main()
{
    signal(SIGINT, signal_handler);
    signal(SIGTERM, signal_handler);

    printf("Process ID: %d\n", getpid());
    printf("Press Ctrl+C to send SIGINT\n");

    while(1)
    {
        sleep(1);
    }

    return 0;
}
```

RESULT:

```
root@SRIVALLI:~# nano signal_test.c
root@SRIVALLI:~# gcc signal_test.c -o signal_test
root@SRIVALLI:~# ./signal_test
Process ID: 610
Press Ctrl+C to send SIGINT
^C
Received Signal: 2
```

REPORT:

In this experiment, signal delivery was analyzed using a simple signal handling program. The process received signals such as SIGINT and SIGTERM and responded through a signal handler. The experiment helped in understanding signal delivery, process behavior, terminal-generated signals, and process group communication in Linux systems.

4. To implement how to resume Stopped Jobs, Send Continue Signals, Update Job Status, Maintain Job Table, Verify Execution, Test Background Recovery

CODE:

```

GNU nano 7.2 resume_job.c
#include <stdio.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

int main()
{
    pid_t pid = fork();

    if(pid == 0)
    {
        printf("Child Process Running...\n");

        for(int i = 1; i <= 10; i++)
        {
            printf("Count = %d\n", i);
            sleep(1);
        }
    }
    else
    {
        sleep(3);

        kill(pid, SIGSTOP);
        printf("Job Stopped\n");

        sleep(3);

        kill(pid, SIGCONT);
        printf("Job Resumed\n");

        wait(NULL);

        printf("Job Status: Completed\n");
    }

    return 0;
}

```

RESULT:

```

root@SRIVALLI:~# nano resume_job.c
root@SRIVALLI:~# gcc resume_job.c -o resume_job
root@SRIVALLI:~# ./resume_job
Child Process Running...
Count = 1
Count = 2
Count = 3
Job Stopped
Job Resumed
Count = 4
Count = 5
Count = 6
Count = 7
Count = 8
Count = 9
Count = 10
Job Status: Completed

```

REPORT:

In this experiment, a stopped job was resumed using the SIGCONT signal. The child process was first stopped using SIGSTOP and later resumed for execution. The job

status was updated and monitored until completion. This experiment helps in understanding job control, signal handling, background job recovery, and process management in Linux systems.